



St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Academic Year: 2025-2026 Term: Even (Jan. 2026 – Jun. 2026) Class / Branch: SE – AIML

Semester: IV

Course: Web Programming Lab. (AI4VS_LR4)

Date of Assignment: / /2026 Date of Submission: / /2026

Pre-Lab Exercises for Experiment-10

Exercise 1: Understanding Node.js Environment

Task:

1. Install Node.js.
2. Verify installation using:
 - a. `node -v`
 - b. `npm -v`
3. Create a folder named practice-node.
4. Initialize npm inside it.
5. Identify the purpose of and explain :

- a. `package.json`

Ans. **package.json** is a configuration file for a Node.js project. It contains project information such as name, version, dependencies, scripts and metadata required to manage the application.

- b. `node_modules`

Ans. **node_modules** is a folder automatically created when npm installs packages. It stores all the libraries and modules required by the project.

- c. `dependencies`

Ans. **Dependencies** are external packages or libraries that a project needs to run properly. They are listed inside `package.json` and installed inside the `node_modules` folder.

OUTPUT:

```
C:\Users\Angel>node -v
v24.13.1

C:\Users\Angel>npm -v
11.8.0

C:\Users\Angel>mkdir practice-node

C:\Users\Angel>cd practice-node

C:\Users\Angel\practice-node>npm init -y
Wrote to C:\Users\Angel\practice-node\package.json:
```

```
{
  "name": "practice-node",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Exercise 2: Create a Basic Node Server (Without Express)

Task:

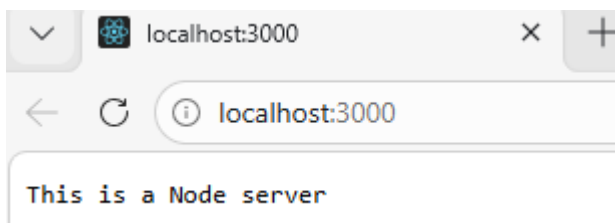
1. Create a simple HTTP server using Node's built-in http module.
2. Make it respond with:
 - a. "This is a Node server"
3. Run the server.
4. Access it in the browser using localhost.

CODE

```
index.js
const http = require("http");
const server = http.createServer((req,
res) => {
  res.write("This is a Node
server");
  res.end();
});
server.listen(3000, () => {
  console.log("Server running at
http://localhost:3000");
});
```

OUTPUT

```
C:\Users\Angel\practice-node>node index.js
Server running at http://localhost:3000
```



Observation:

- What happens when you refresh the browser?
Ans. When the browser is refreshed, the browser sends a new HTTP request to the Node server. The server processes the request again and returns the same response message **"This is a Node server."**
- What happens if you change the port?
Ans. If the port number in the code is changed (eg. **3000 to 4000**), the server will start on the new port. The user must access the server using **http://localhost:4000** instead of **http://localhost:3000**.

Exercise 3: Install and Explore Express

Task:

1. Create a new folder express-practice.
2. Initialize npm.
3. Install Express.
4. Observe and explain

- a. Changes in package.json

Ans. After installing Express using **npm install express**, the **package.json** file is automatically updated. A new section called **dependencies** is added where Express is listed along with its version. This indicates that the project depends on the Express framework to run.

- b. Creation of node_modules

Ans. When Express is installed, a folder called **node_modules** is automatically created. This folder contains the Express package and all other libraries required by it. It stores all the installed dependencies used in the project.

5. Write down:

- a. What is Express?

Ans. Express is a **fast and lightweight web application framework for Node.js** used to build web servers and APIs. It provides useful features like routing, middleware support, and simplified request-response handling.

- b. Why is it used?

Ans. Express is used because it simplifies server-side development in Node.js. It helps in creating web applications and APIs easily, managing routes efficiently, and handling HTTP requests and responses with less code.

OUTPUT:

```
C:\Users\Angel>cd express-practice

C:\Users\Angel\express-practice>npm init -y
Wrote to C:\Users\Angel\express-practice\package.json:

{
  "name": "express-practice",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
C:\Users\Angel\express-practice>npm install express

added 65 packages, and audited 66 packages in 4s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
C:\Users\Angel\express-practice>npm init -y
Wrote to C:\Users\Angel\express-practice\package.json:

{
  "name": "express-practice",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {}
}
```

Exercise 4: Create Simple Routes

Task:

1. Create an Express server.
2. Create the following routes:
 - a. / → Send text response
 - b. /contact → Send text response
 - c. /help → Send text response
3. Test each route in the browser.

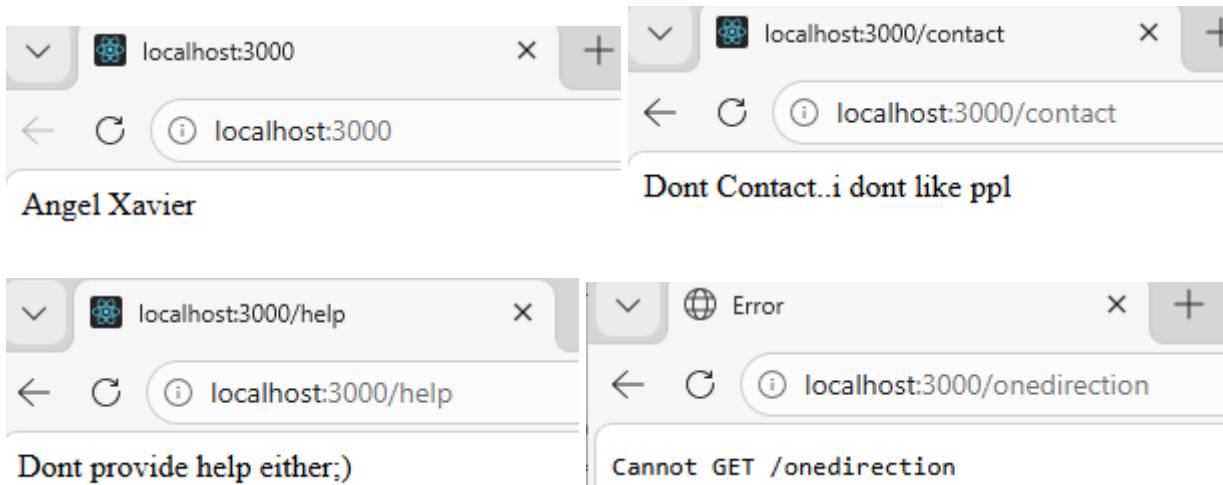
CODE

server.js

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  res.send("Angel Xavier");
});
app.get("/contact", (req, res) => {
  res.send("Dont Contact..i dont like ppl");
});
```

```
app.get("/help", (req, res) => {
  res.send("Dont provide help either;");
});
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT



Questions:

- What happens if you type a route that is not defined?
Ans. If a user enters a route that is not defined, Express returns **"Cannot GET /route-name"** which indicates that the requested route does not exist.
- How does Express differentiate routes?
Ans. Express differentiates routes based on the **URL path and HTTP method**. It matches the incoming request path with the route defined in the code and executes the corresponding handler.

Exercise 5: Return JSON Data

Task:

1. Create a route `/api/user`.
2. Return JSON data with:
 - a. name
 - b. age
 - c. course
3. Test using:
 - a. Browser
 - b. Postman

CODE

server.js

```
const express = require("express");
const app = express();
app.get("/", (req, res) => {
  res.send("Angel Xavier");
});
app.get("/api/user", (req, res) => {
  res.json({
    name: "Angel Xavier",
    age: 20,
    course: "AIML"
  });
});
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

OUTPUT

The screenshot shows a web browser with a REST client interface. The URL bar shows `localhost:3000/api/user`. The response is a JSON object: `{ "name": "Angel Xavier", "age": 20, "course": "AIML" }`. The status is `200 OK`. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, Tests, and Settings. The Body tab is selected, showing the JSON response. The response is also displayed in a table format below the JSON view.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Questions:

- What is the format of JSON?
Ans. JSON (JavaScript Object Notation) is a lightweight data format used for data exchange. It stores data in **key-value pairs** and uses **curly braces { }** to represent objects. Keys are written as strings and values can be strings, numbers, arrays, or objects.
- How is JSON different from plain text?
Ans. JSON is a **structured data format** where information is organized in key-value pairs, making it easy for programs to read and process. Plain text is **unstructured** and only contains simple characters or sentences without any defined format.

Exercise 6: Middleware Practice

Task:

1. Create middleware that logs:
 - a. Request method
 - b. Request URL
2. Attach it to the application.
3. Access different routes.

Observation:

- Does middleware run for every request?
Ans. Yes, middleware runs for every incoming request because it is attached to the application using `app.use()`.
- When does it execute?
Ans. Middleware executes before the request reaches the route handler. It processes the request first and then passes control to the next function using `next()`.

CODE

```
server.js
const express = require("express");
const app = express();
// Middleware
app.use((req, res, next) => {
  console.log("Method:",
req.method);
  console.log("URL:", req.url);
  next();
});
app.get("/", (req, res) => {
  res.send("Angel Xavier");
});
app.get("/contact", (req, res) => {
  res.send("Dont Contact..i dont
like ppl");
});

app.get("/help", (req, res) => {
  res.send("Dont provide help
either;");
});
app.get("/api/user", (req, res) => {
  res.json({
    name: "Angel Xavier",
    age: 20,
    course: "AIML"
  });
});
app.listen(3000, () => {
  console.log("Server running at
http://localhost:3000");
});
```

OUTPUT

```
C:\Users\Angel\express-practice>node server.js
Server running at http://localhost:3000
Method: GET
URL: /
Method: GET
URL: /contact
```

Exercise 7: nodemon Practice

Task:

1. Install nodemon.
2. Run your Express server using nodemon
3. Modify your server file.
4. Observe behavior.

OUTPUT

```
C:\Users\Angel\express-practice>npm install -g nodemon  
  
added 28 packages in 3s  
  
5 packages are looking for funding  
  run 'npm fund' for details
```

```
C:\Users\Angel\express-practice>nodemon server.js  
[nodemon] 3.1.14  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node server.js`  
Server running at http://localhost:3000  
Method: GET  
URL: /  
Method: GET  
URL: /favicon.ico  
[nodemon] restarting due to changes...  
[nodemon] starting `node server.js`  
Server running at http://localhost:3000  
Method: GET  
URL: /
```

Questions:

- What is the difference between running with node and nodemon?
Ans. When running with node, the server must be manually restarted every time the code is changed. With nodemon, the server automatically restarts whenever a file is modified.
- Why is nodemon useful during development?
Ans. Nodemon is useful because it automatically detects changes in the project files and restarts the server, saving time and making development faster and more efficient.

Exercise 8: 404 Handling

Task:

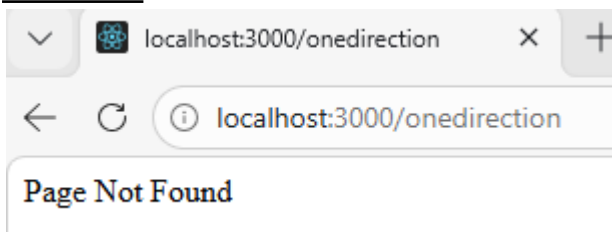
1. Create a handler for undefined routes.
2. Return:
 - a. "Page Not Found"
3. Test with an invalid URL.

CODE:

```
server.js
const express = require("express");
const app = express();
app.get("/", (req, res) => {
    res.send("Angel Xavier");
});
app.get("/contact", (req, res) => {
    res.send("Contact Page");
});
app.get("/help", (req, res) => {
    res.send("Help Page");
});

// 404 Handler
app.use((req, res) => {
    res.status(404).send("Page Not Found");
});
app.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

OUTPUT:



Question:

- Why is 404 handling important in backend development?
Ans. 404 handling is important because it informs users when they try to access a route or resource that does not exist. It improves user experience and helps developers manage invalid requests properly instead of leaving the user with an unclear error.

Exercise 9: Request–Response Analysis

After completing all exercises, write:

1. Step-by-step explanation of what happens when:

a. A user types localhost:3000/ in browser.

Ans.

- The user enters <http://localhost:3000/> in the browser.
- The browser sends an **HTTP request** to the Node.js server running on port **3000**.
- The request first passes through the **middleware** in the Express application.
- The middleware processes the request (for example, logging the request method and URL).
- After middleware execution, Express checks the **routes defined in the server**.
- When the matching route **/** is found, the corresponding route handler executes.
- The server sends a **response** back to the browser using **res.send()**.
- The browser receives the response and displays the message on the webpage.

2. Explain:

a. What is a request?

Ans. A **request** is a message sent by the client (browser or application) to the server asking for a resource or service. It contains information such as the request method (GET, POST), URL, headers, and other data.

b. What is a response?

Ans. A **response** is the message sent by the server back to the client after processing the request. It contains the requested data, status code, and other information that the browser displays.

c. Where does middleware execute?

Ans. Middleware executes **between the incoming request and the route handler**. It runs before the request reaches the final route and can process or modify the request and response before passing control to the next function.